The Essential of Compilers

Lectures 1 to 5 in one read: the lexer as a refresher, top-down parsing in depth, every mock question placed where it is answered

Compilers

2026-10-31

How to read this document

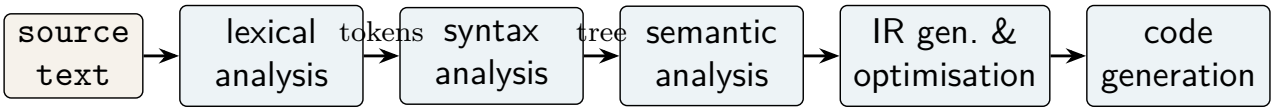
Part I (sections 1–5) is the lexical half of the course, compressed to the definitions, the four constructions and the scanner loop that exam questions use. Part II (sections 6–10) is Lecture 5, the new material, and gets about two thirds of the pages: grammars and trees, recursive descent, FIRST and FOLLOW, the LL(1) table and stack machine, and the transformations that make a grammar LL(1). Every term is defined before it is used. Each section ends with a *Mock drill* box: the mock-midterm questions that live in that section, answered the way the rubric wants, plus a new question of the same kind. Every set, table and trace below was recomputed by `check_ll1.py`.

Mock question	Topic	Section
1.1	definition of a CFG, Chomsky types	1, 6
1.2	$a^n b^{2n}$ not regular; its grammar	4
1.3	leftmost/rightmost derivation, parse tree	6
1.4	ambiguity, associativity, LL(1) rewrite	6, 10
1.5	FIRST/FOLLOW, which cell conflicts	8, 9
1.6	eliminating ε-productions	10
1.7	NFA vs DFA blow-up; number regex	2, 3
2.1, 2.2, 2.4	scanner trace, combined DFA, rule order	5
2.3	comment regex and DFA; nested comments	4
3	left recursion, FIRST/FOLLOW, table	8, 9, 10
4	recursive descent, calls, stack trace, error detection	7, 9
5	FIRST/FOLLOW with nullable chains, table, trace	8, 9

Part I. The lexer, as a refresher

1 What a Compiler Does, and the Hierarchy Behind It

A compiler translates a program in a source language into an equivalent program in a target language, and reports errors in the source. It works in **phases**, each consuming the previous phase's output:



The **lexer** (scanner) groups characters into **tokens** and drops whitespace and comments. The **parser** checks that the token sequence has the structure of a program and builds a parse tree. Semantic analysis checks what the grammar cannot (types, declarations); the back end produces code. This course covers the first two phases, and they correspond to the two lowest levels of the Chomsky hierarchy:

Type	Production shape	Recogniser	In the compiler
3, regular	$A \to aB$ or $A \to a$ (or a regex)	finite automaton (DFA/NFA)	tokens: the lexer
2, context-free	$A \to w$, one nonterminal on the left	pushdown automaton (stack)	program structure: the parser
1, context-sensitive	$\alpha \to \beta$ with $\lvert\alpha\rvert \le \lvert\beta\rvert$	linear-bounded automaton	(declared-before-use, types)
0, unrestricted	$\alpha \to \beta$	Turing machine	—

Each class strictly contains the one above it in the table. The split matters for design: tokens need no memory beyond a fixed number of states, so a DFA is enough and very fast; nesting (parentheses, blocks, nested comments) needs unbounded memory, which a stack provides.

Mock drill

Mock 1.1. *Which statement describes a CFG $G = (N, T, S, P)$?* **(A)**: every production is $A \to w$, a single nonterminal on the left and any string $w \in (N \cup T)^*$, possibly empty, on the right. (B) is type 1 (context-sensitive), (C) is type 3 (right-linear, regular), (D) confuses a *class* (shape of the rules) with a *property*: the CFG $E \to E - E \mid \mathsf{NUM}$ is ambiguous.

2 Tokens and Regular Expressions

Concept: Token, lexeme, pattern

A **token** is a category with a name (ID, INT, LE) and often an attribute. A **lexeme** is the actual character string that was matched (`x12`, `42`, `<=`). A **pattern** is the rule that describes which lexemes belong to the token, written as a regular expression.

Regular expressions over an alphabet Σ: $\emptyset$, ε, a symbol a; and, if R, S are regular expressions, $R \mid S$ (union), RS (concatenation), R^* (zero or more). Precedence from strongest to weakest: star, concatenation, union, so $ab^* \mid c$ is $(a(b^*)) \mid c$. Shorthands: $R^+ = RR^*$, $R? = R \mid \varepsilon$, character classes `[0-9]`, `[a-z]`.

Method: Building a token regex that the grader accepts

(1) Split the lexeme into its parts and decide for each whether it is mandatory, optional (`?`) or repeated (`*`, `+`). (2) Write one sub-expression per part. (3) Test against *every* given example, and check that each must-not-match example fails, and say which part makes it fail.

Mock drill

Mock 1.7(b). Integer part, optional fraction, optional exponent, with $D = $ `[0-9]`:

$$D^+ \; (\; . \; D^+ \;)? \; (\; (\texttt{E} \mid \texttt{e}) \; (\texttt{+} \mid \texttt{-})? \; D^+ \;)?$$

`42`, `3.14`, `6.02E23`, `1.5e-3` match. `.5` fails because D^+ at the front is mandatory; `3.` fails because a dot must be followed by D^+; `1e` fails because an exponent needs D^+. Saying *why* each counter-example fails is what earns the second point.

New. Identifiers of letters, digits and underscores that start with a letter and do not end with an underscore: $L \; (\; (L \mid D \mid _)^* \; (L \mid D) \;)?$. The single letter is covered by the optional group being absent.

3 From a Regular Expression to a Minimal DFA

Concept: NFA and DFA

A **DFA** has, from every state, *at most one* transition per input symbol and no ε-transitions; running it is a table lookup per character. An **NFA** may have several transitions on the same symbol and ε-transitions; it accepts a string if *some* path reaches an accepting state. Both recognise exactly the regular languages.

The lexer generator follows a fixed pipeline: **regex** $\rightarrow$ **NFA** $\rightarrow$ **DFA** $\rightarrow$ **minimal DFA**.

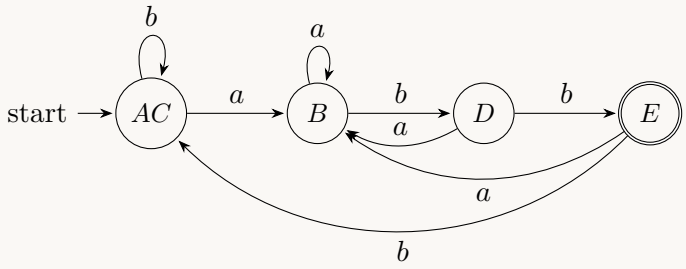
Method 1, Thompson's construction (regex $\rightarrow$ NFA). Each piece gets an NFA with one start and one accepting state. A symbol a: two states, one a-edge. RS: the accepting state of R is linked by ε to the start of S. $R \mid S$: a new start with ε-edges to both starts, and both ends join a new accepting state by ε. R^*: new start and end; ε from start to end (zero times), ε into R, and ε from R's end back to R's start (again) and to the new end. Every operator adds at most two states, so the NFA has at most $2\times$(number of symbols and operators) states.

Method 2, subset construction (NFA $\rightarrow$ DFA). A DFA state is the *set* of NFA states the NFA could be in. Start with ε-closure$(\{s_0\})$. For a DFA state T and symbol a, the successor is ε-closure(move(T, a)). Repeat until no new set appears; a set is accepting if it contains an accepting NFA state.

Method 3, minimisation (partition refinement). Start from two groups, accepting and non-accepting. Split a group whenever two of its states go, on some symbol, to different groups. When nothing splits, each group is one state of the minimal DFA.

Worked example: $(a \mid b)^*abb$, the classic

Thompson gives an 11-state NFA (states 0–10). The subset construction gives five DFA states, $A = \{0, 1, 2, 4, 7\}$, $B = \{1, 2, 3, 4, 6, 7, 8\}$, $C = \{1, 2, 4, 5, 6, 7\}$, $D = \{1, 2, 4, 5, 6, 7, 9\}$, $E = \{1, 2, 4, 5, 6, 7, 10\}$, with E accepting. Transitions (a, b): $A \rightarrow (B, C)$, $B \rightarrow (B, D)$, $C \rightarrow (B, C)$, $D \rightarrow (B, E)$, $E \rightarrow (B, C)$. Minimisation: $\{A, B, C, D\}, \{E\}$; on b, D goes to E and the others do not: $\{A, B, C\}, \{D\}, \{E\}$; on b, B goes to D and A, C go to C: $\{A, C\}, \{B\}, \{D\}, \{E\}$. No further split: 4 states.



Each state remembers how much of the suffix abb has just been read: nothing, a, ab, abb.

Mock drill

Mock 1.7(a). (D): every NFA converts to an equivalent DFA, but the DFA's states are *sets* of NFA states, so there can be up to 2^n. The blow-up is real: "the n-th symbol from the end is a" has an $(n{+}1)$-state NFA and needs 2^n DFA states, because the DFA must remember the last n symbols. (A) false: same power; generators prefer DFAs because they never backtrack among paths. (B) is backwards. (C) false: the count can drop (unreachable subsets) or explode.

4 What a Finite Automaton Cannot Do

Method: Proving a language is not regular: distinguishable strings

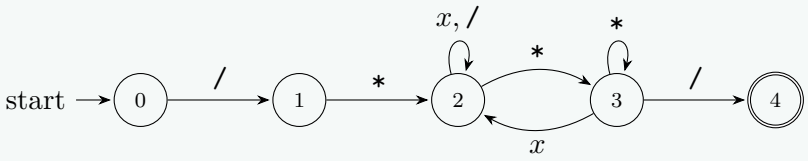
Two strings u, v are **distinguishable** for L if some suffix z puts exactly one of uz, vz in L. A DFA must send distinguishable strings to different states (otherwise it treats uz and vz the same). So if L has infinitely many pairwise distinguishable strings, no DFA with finitely many states accepts it.

The pumping lemma says the same thing: in any accepted string longer than the number of states, some state repeats, so a middle part y can be repeated (xy^iz) without the DFA noticing.

Mock drill

Mock 1.2. $L = \{a^n b^{2n}\}$. **(a)** The strings $a^0, a^1, a^2, \ldots$ are pairwise distinguishable: for $i \neq j$ the suffix b^{2i} accepts $a^i b^{2i}$ and rejects $a^j b^{2i}$. Infinitely many classes, so no DFA. (Pumping: $w = a^p b^{2p}$, $y = a^k$ inside the first p symbols, $xy^2z = a^{p+k}b^{2p} \notin L$.) **(b)** $S \to a\,S\,bb \mid \varepsilon$.

Mock 2.3, comments over $\{/, *, x\}$. **(a)** `/*` $(x \mid /\mid *^+x)^*$ `*`$^+$`/`: inside the body, a run of stars may only be followed by x (a `/` would close the comment), and the closing is `*`$^+$`/`, which also covers `**/`. **(b)**



State 2: inside, last symbol not a star; state 3: inside, just read stars; state 4: the first `*/` was read and has no outgoing edges. **(c)** Nested comments: no DFA. The prefixes $(/*)^1, (/*)^2, \ldots$ are pairwise distinguishable (they need different numbers of `*/` to close). It is the balanced-parenthesis language in disguise, so the lexer needs a depth counter (flex: a start condition plus

a variable).

5 The Scanner: Many Rules, One DFA, Longest Match

A lexer specification is an ordered list of rules `pattern  action`. The generator builds one NFA per rule, joins them under a new start state by ε-edges, and runs the subset construction on the whole thing. A DFA state may then contain the accepting states of several rules.

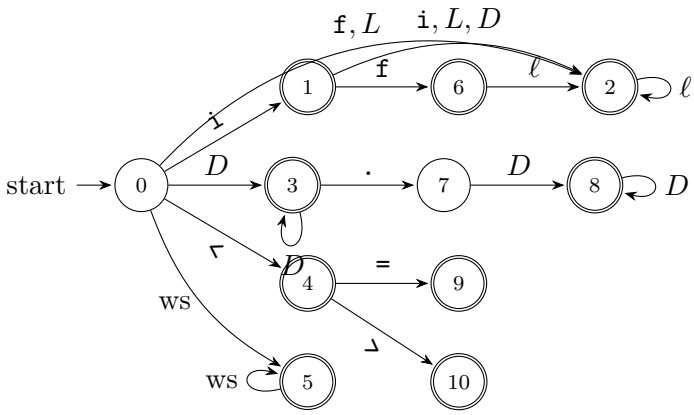
Concept: The two disambiguation rules

Longest match (maximal munch). Among all prefixes of the remaining input that some rule matches, take the longest. **Priority.** If several rules match that longest prefix, the rule *listed first* wins. In the DFA, an accepting state is labelled with the first-listed rule among those it accepts.

Mechanism: The scan loop with backup

From the start state, keep following transitions while one exists, and remember the **last accepting state** passed and the input position at that moment. When stuck (no transition, or end of input), **back up** to that position, emit that state's token, and restart in the start state. If no accepting state was passed, report a lexical error on the first character and skip it.

Backing up has a cost: with the rules `a` and `a*b` on the input a^n, every token reads to the end of the input hoping for a `b`, then backs up to a single `a`: about $n^2/2$ character reads.



1, 2: ID 6: **IF**
3: INT 8: REAL
4: LT 9: LE 10: NE
5: skip 7: not accepting
$\ell = \{\texttt{i}, \texttt{f}, L, D\}$

Mock drill

Mock 2.2. The DFA above is the combined automaton for the eight rules (`if`, identifier, int, real, `<`, `<=`, `<>`, whitespace). State 6 is reached by `if` and accepts both the keyword and the identifier rule; it is labelled IF because `if` is listed first. One more letter or digit goes to state 2, identifier only. Longest match is the scan loop with backup described above.

Mock 2.1, input `if iff<=3.14<>x12 if3 7.`: IF, ID(iff), LE, REAL(3.14), NE, ID(x12), ID(if3), INT(7), then a lexical error on the final `.`. On `7.` the scanner reached state 7 (not accepting), hit the end of input, backed up to state 3 and emitted INT; `.` cannot start any token.

Mock 2.4. Identifier rule moved above `if`: **(A)**, ID for both. On `if` the two rules tie in length and the first listed now wins; on `iff` only the identifier matches 3 characters. The keyword rule

can never fire again (flex warns "rule cannot be matched"). Order matters exactly on ties, never otherwise.

Part II. Parsing (Lecture 5), in depth

6 Grammars, Derivations, Trees and Ambiguity

Concept: Context-free grammar

$G = (N, T, S, P)$: nonterminals N, terminals T (the tokens), a start symbol $S \in N$, and productions $A \to \alpha$ with $A \in N$ and $\alpha \in (N \cup T)^*$. Conventions: capitals are nonterminals, lower case and quoted symbols are terminals, Greek letters α, β, γ are strings of either, ε is the empty string, $A \to \alpha \mid \beta$ abbreviates two productions.

A **derivation step** $\gamma A \delta \Rightarrow \gamma \alpha \delta$ replaces one nonterminal by one of its right sides; $\Rightarrow^*$ is zero or more steps. Every string derivable from S is a **sentential form**; one made only of terminals is a **sentence**, and the sentences form the language $L(G)$. A **leftmost** derivation ($\underset{lm}{\Rightarrow}$) always replaces the leftmost nonterminal; a **rightmost** one ($\underset{rm}{\Rightarrow}$), the rightmost. A **parse tree** has S at the root, a production at each interior node, and the sentence in its leaves read left to right. It records *which* productions were used, but not the *order*: one tree corresponds to exactly one leftmost and one rightmost derivation.

Concept: Ambiguity

G is **ambiguous** if some sentence has two different parse trees (equivalently, two different leftmost derivations). Ambiguity is a property of the *grammar*: the same language often has an unambiguous grammar. An ambiguous grammar is never LL(1), since some cell of the table must offer two ways to build the tree.

Technique: Precedence and associativity by layering

One nonterminal per precedence level, the loosest at the top; the operator's recursion side gives its associativity:

$$E \to E + T \mid E - T \mid T, \qquad T \to T * F \mid T/F \mid F, \qquad F \to (E) \mid \mathsf{num}.$$

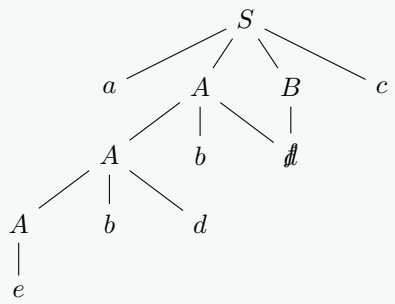
$E \to E - T$ is *left*-recursive, so $a - b - c$ can only be grouped as $(a - b) - c$: left-associative. A right-associative operator (exponentiation) is written right-recursive, $P \to F \hat{} P \mid F$. Since $*$ sits one level lower, $a + b * c$ can only group $b * c$ first.

Mock drill

Mock 1.3, $S \to aABc$, $A \to Abd \mid e$, $B \to f$, sentence a e b d b d f c. A must produce e b d b d, i.e. $A \Rightarrow Abd \Rightarrow Abdbd \Rightarrow ebdbd$.

$$\text{leftmost: } S \underset{lm}{\Rightarrow} aABc \underset{lm}{\Rightarrow} aAbdBc \underset{lm}{\Rightarrow} aAbdbdBc \underset{lm}{\Rightarrow} aebdbdBc \underset{lm}{\Rightarrow} aebdbdfc$$

$$\text{rightmost: } S \underset{rm}{\Rightarrow} aABc \underset{rm}{\Rightarrow} aAfc \underset{rm}{\Rightarrow} aAbdfc \underset{rm}{\Rightarrow} aAbdbdfc \underset{rm}{\Rightarrow} aebdbdfc$$



Mock 1.4. $E \to E - E \mid (E) \mid$ NUM. (a) NUM - NUM - NUM has the tree grouping the first two and the tree grouping the last two. (b) For $10 - 4 - 3$: $(10 - 4) - 3 = 3$ and $10 - (4 - 3) = 9$; the compiler needs the first, **left associativity**. (c) $E \to E - T \mid T$, $T \to (E) \mid$ NUM. (d) see §10: $E \to TE'$, $E' \to -TE' \mid \varepsilon$, $T \to (E) \mid$ NUM; the left-recursive version makes `E()` call itself before consuming a token.

7 Top-Down Parsing and Recursive Descent

A top-down parser builds the tree from the root, producing a *leftmost* derivation. At each step it faces one question:

> The leftmost nonterminal is A and the next token (the **lookahead**) is a. Which production of A do I use?

Guessing and undoing wrong guesses (backtracking) works but can take exponential time. A **predictive** parser answers from the lookahead alone, never undoing anything. The rest of the chapter computes, from the grammar, a table of these answers.

Method: Writing a recursive-descent parser from a grammar

One procedure per nonterminal. In `A()`, test the lookahead against the *selection set* of each production of A (§9) and run the chosen right side: for a terminal t, `match(t)` (check that the lookahead is t, then advance); for a nonterminal B, call `B()`. An ε-production is "do nothing and return". If no selection set contains the lookahead, `error()`.

```
void Eprime() {                          /* E' -> + T E'  |  eps */
    if (lookahead == PLUS) { match(PLUS); T(); Eprime(); }
    else if (lookahead == SEMI || lookahead == RPAREN) { /* eps: FOLLOW(E') */ }
    else error();
}
```

Two remarks the mock tests. First, many hand-written parsers make the ε branch the default ("otherwise return") with no `error()`: that accepts the same language but reports errors *later*, at the next `match`. Second, a production $A \to A\alpha$ becomes `A()` whose first act is to call `A()` with the same lookahead: infinite recursion. That is why left recursion must go (§10).

Mock drill

Mock 4.1, reading the grammar off the code. Each `if` branch is one production; its condition is the selection set: $S \to$ ID := E ; S {ID} | PRINT E ; S {PRINT} | ε {EOF}; $E \to TE'$

{ID,NUM,(}; $E' \to +TE'$ {+} | ε FOLLOW(E') = {;,)}; $T \to$ ID | NUM | (E).

Mock 4.2(b), the call sequence for `id := ( num + id ) ;`: `S, match(ID), match(ASSIGN), E, T, match(LPAREN), E, T, match(NUM), Eprime, match(PLUS), T, match(ID), Eprime` (sees), returns), `match(RPAREN), Eprime` (sees ;, returns), `match(SEMI), S`.

Mock 4.5. `void E(){ E(); match(PLUS); T(); }`: **(B)**, `E()` calls itself without consuming a token and the stack overflows. (C) is wrong because `match(PLUS)` is never reached; (D) because nothing is reported, the program crashes.

8 Nullable, FIRST and FOLLOW

To fill the table we need two facts about the grammar: which tokens can *start* what a string derives, and which tokens can come *right after* a nonterminal when it derives nothing.

Concept: Nullable

A is **nullable** if $A \Rightarrow^* \varepsilon$. Find them by fixed point: mark A if it has $A \to \varepsilon$, or a production whose symbols are all already marked; repeat until nothing changes. A string is nullable iff all its symbols are; a terminal never is.

Concept: FIRST

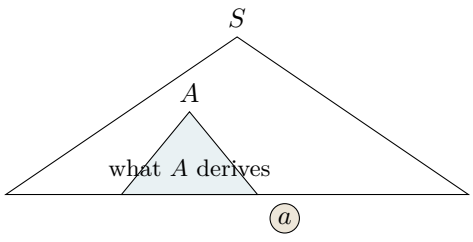
FIRST(α) is the set of terminals that can begin a string derived from α, plus ε if α is nullable. The ε is a flag ("can vanish"), not a token.

Method: Computing FIRST

FIRST(t) = {t} for a terminal. For $X \to Y_1 Y_2 \cdots Y_k$: add FIRST(Y_1) \ {ε}; if Y_1 is nullable, also FIRST(Y_2) \ {ε}; continue while the symbols are nullable; **stop at the first non-nullable symbol**; if all Y_i are nullable, add ε. $X \to \varepsilon$ adds ε. Repeat over all productions until no set changes.

Concept: FOLLOW

FOLLOW(A) is the set of terminals that can appear immediately to the right of A in some sentential form, with \$ (end of input) included when A can be last. It never contains ε.



a is the first token to the right of A's subtree. If A derives ε, the subtree is empty and a is what the parser sees as lookahead while standing at A.

Method: Computing FOLLOW

(1) $\$ \in \text{FOLLOW}(S)$. (2) For every occurrence $B \to \alpha A \beta$: add $\text{FIRST}(\beta) \setminus \{\varepsilon\}$ to $\text{FOLLOW}(A)$. (3) If β is nullable or empty (A is last): add $\text{FOLLOW}(B)$ to $\text{FOLLOW}(A)$. Repeat until no set changes. Look at *every occurrence* of A on a right side, not at A's own productions.

Worked example: The expression grammar, start to finish

$$1\ E \to TE' \qquad 2\ E' \to +TE' \qquad 3\ E' \to \varepsilon \qquad 4\ T \to FT'$$
$$5\ T' \to *FT' \qquad 6\ T' \to \varepsilon \qquad 7\ F \to (E) \qquad 8\ F \to \mathsf{id}$$

Nullable: E', T'. FIRST, from the bottom up: F: $\{(, \mathsf{id}\}$; T': $\{*, \varepsilon\}$; T: F is not nullable, stop: $\{(, \mathsf{id}\}$; E': $\{+, \varepsilon\}$; E: $\{(, \mathsf{id}\}$.
FOLLOW, top down:

Set	Where it comes from	Result
$\text{FOLLOW}(E)$	start symbol: \$; rule 7:)	$\{\$,)\}$
$\text{FOLLOW}(E')$	last in rules 1 and 2: $\text{FOLLOW}(E)$	$\{\$,)\}$
$\text{FOLLOW}(T)$	before E' in 1, 2: $\text{FIRST}(E') \setminus \varepsilon = \{+\}$; E' nullable: $\text{FOLLOW}(E)$	$\{+, \$,)\}$
$\text{FOLLOW}(T')$	last in rules 4 and 5: $\text{FOLLOW}(T)$	$\{+, \$,)\}$
$\text{FOLLOW}(F)$	before T': $\{*\}$; T' nullable: $\text{FOLLOW}(T)$	$\{*, +, \$,)\}$

Warning / Gotcha: The four classic FIRST/FOLLOW errors

(1) Stopping the FIRST scan too early (forgetting Y_1 can vanish) or too late (continuing past a non-nullable symbol: $\varepsilon \notin \text{FIRST}(T)$ although T' is nullable). (2) Putting ε in a FOLLOW set. (3) Computing FOLLOW from A's own productions instead of from where A *occurs*. (4) Forgetting the chain: when β is nullable, $\text{FOLLOW}(B)$ flows into $\text{FOLLOW}(A)$, and through two nullable symbols in a row (mock 5: $\text{FOLLOW}(P) \supseteq \text{FIRST}(Q) \cup \text{FIRST}(R) \cup \text{FOLLOW}(S)$).

Mock drill

Mock 1.5(a), $S \to Ax \mid Bx$, $A \to a \mid \varepsilon$, $B \to b \mid \varepsilon$: $\text{FIRST}(A) = \{a, \varepsilon\}$, $\text{FIRST}(B) = \{b, \varepsilon\}$, $\text{FIRST}(S) = \{a, b, x\}$ (x because A, B can vanish); $\text{FOLLOW}(S) = \{\$\}$, $\text{FOLLOW}(A) = \text{FOLLOW}(B) = \{x\}$.

Mock 5(a), $S \to PQR$, $P \to (S) \mid xSy \mid z$, $Q \to xQ \mid \varepsilon$, $R \to wR \mid \varepsilon$. Nullable: Q, R.

	FIRST	FOLLOW
S	$\{(, x, z\}$	$\{\$,), y\}$: start; after S in (S) and xSy
P	$\{(, x, z\}$	$\{x, w, \$,), y\}$: $\text{FIRST}(Q) \cup \text{FIRST}(R)$ minus ε, and QR nullable so $\text{FOLLOW}(S)$
Q	$\{x, \varepsilon\}$	$\{w, \$,), y\}$: $\text{FIRST}(R) \setminus \varepsilon$, R nullable so $\text{FOLLOW}(S)$
R	$\{w, \varepsilon\}$	$\{\$,), y\}$: last in $S \to PQR$

New, a nullable-chain drill. $S \to ACB \mid CbB \mid Ba$, $A \to da \mid BC$, $B \to g \mid \varepsilon$, $C \to h \mid \varepsilon$. Answers: $\text{FIRST}(S) = \{d, g, h, b, a, \varepsilon\}$, $\text{FIRST}(A) = \{d, g, h, \varepsilon\}$, $\text{FIRST}(B) = \{g, \varepsilon\}$, $\text{FIRST}(C) = \{h, \varepsilon\}$; $\text{FOLLOW}(S) = \{\$\}$, $\text{FOLLOW}(A) = \{h, g, \$\}$, $\text{FOLLOW}(B) = \{a, \$, h, g\}$, $\text{FOLLOW}(C) = \{g, \$, b, h\}$. It is *not* LL(1): conflicts at $M[S, g]$, $M[S, h]$, $M[B, g]$, $M[C, h]$.

9 The LL(1) Table, the LL(1) Condition and the Stack Machine

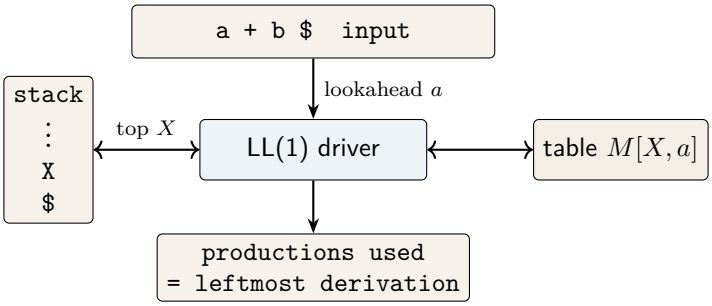
LL(1): the input is read **L**eft to right, the parser builds a **L**eftmost derivation, with **1** token of lookahead.

Method: Filling the table $M[A, a]$

For each production $A \to \alpha$ (numbered top to bottom): (1) for every terminal $a \in \text{FIRST}(\alpha)$, put the production in $M[A, a]$; (2) if α is nullable ($\varepsilon \in \text{FIRST}(\alpha)$), also put it in $M[A, b]$ for every $b \in \text{FOLLOW}(A)$, \$ included. Empty cells mean syntax error. The set of tokens that selects $A \to \alpha$ is its **selection set**: $\text{FIRST}(\alpha) \setminus \{\varepsilon\}$, plus $\text{FOLLOW}(A)$ if α is nullable.

Concept: The LL(1) condition

G is LL(1) iff **no cell holds two productions**, i.e. for any two productions $A \to \alpha \mid \beta$: **FIRST/FIRST**: $\text{FIRST}(\alpha) \cap \text{FIRST}(\beta) = \emptyset$ (at most one of them nullable); **FIRST/FOLLOW**: if β is nullable, $\text{FIRST}(\alpha) \cap \text{FOLLOW}(A) = \emptyset$.



Mechanism: The stack algorithm

Start with stack \$ S (top at the right). Repeat, with X the top and a the lookahead:

- $X = a = \$$: **accept**.
- X is a terminal: if $X = a$, pop and advance (**match**); otherwise error.
- X is a nonterminal: if $M[X, a]$ holds $X \to Y_1 \cdots Y_k$, pop X and push $Y_k, \ldots, Y_1$ (so Y_1 ends on top); for $X \to \varepsilon$, just pop. If the cell is empty: error.

The stack holds exactly the part of the sentential form still to be matched, so the table parser and the recursive-descent parser are the same machine: the stack is the call stack made explicit, and a cell is the `if` branch that the procedure would take.

Worked example: Table and trace for id + id * id

	id	+	*	(	)	$
E	1			1		
E'		2			3	3
T	4			4		
T'		6	5		6	6
F	8			7		

Stack	Input	Action
$\$ E$	`id+id*id$`	1
$\$ E'T$	`id+id*id$`	4
$\$ E'T'F$	`id+id*id$`	8
$\$ E'T'$ id	`id+id*id$`	match
$\$ E'T'$	`+id*id$`	6 ($T' \to \varepsilon$)
$\$ E'$	`+id*id$`	2
$\$ E'T+$	`+id*id$`	match
$\$ E'T$	`id*id$`	4, then 8, match
$\$ E'T'$	`*id$`	5
$\$ E'T'F*$	`*id$`	match
$\$ E'T'F$	`id$`	8, match
$\$ E'T'$	`$`	6
$\$ E'$	`$`	3
$\$$	`$`	accept

Productions 3 and 6 sit under FOLLOW sets: that is the second rule of the table construction at work.

Warning / Gotcha: Trace mistakes that cost points

Pushing the right side in the wrong order (the *first* symbol must end on top); forgetting the $ at the bottom of the stack and the end of the input; advancing the input on an expansion (only a match advances); writing "error" when the cell is an ε-production selected by FOLLOW.

Mock drill

Mock 1.5(b), conflict cell: **(C)** $M[S, x]$. $S \to Ax$ is selected on FIRST$(Ax) = \{a, x\}$ and $S \to Bx$ on $\{b, x\}$: both contain x (a FIRST/FIRST conflict through nullable symbols). The language $\{x, ax, bx\}$ is LL(1) with $S \to ax \mid bx \mid x$: the defect is the grammar's.

Mock 3.3 and 5(a), tables. Problem 3, after the fix of §10 (1 *Block* → begin *Stmts* end; 2 *Stmts* → *Stmt Stmts*; 3 *Stmts* → ε; 4 *Stmt* → *Block*; 5 *Stmt* → *Simple*; 6 *Simple* → id ;): $M[Block, \text{begin}] = 1$; $M[Stmts, \text{begin}] = M[Stmts, \text{id}] = 2$, $M[Stmts, \text{end}] = 3$ (FOLLOW); $M[Stmt, \text{begin}] = 4$, $M[Stmt, \text{id}] = 5$; $M[Simple, \text{id}] = 6$. Problem 5 (productions 1 $S \to PQR$, 2–4 P, 5–6 Q, 7–8 R): row S: 1 under $(, x, z$; row P: 2 under (, 3 under x, 4 under z; row Q: 5 under x, 6 under $), y, w, \$$; row R: 7 under w, 8 under $), y, \$$. No cell is doubly filled, so no modification is needed.

Mock 4.4, where errors are caught. `id := num num ;`: the table parser stops at $M[E', \text{NUM}]$ (empty); the code's `Eprime()` returns silently on any non-+ token, so the error appears one step later, in `match(SEMI)` called from `S()`. `id := + ;`: the table stops at $M[E, +]$; the code only notices in `T()`, because `E()` checks nothing before calling `T()`. The table always uses exact selection sets, so it detects an error *no later* than the code.

Mock 4.3 and 5(b) are traces exactly like the example above: expand on the cell, match on equal terminals, ε-productions just pop.

10 When a Grammar Is Not LL(1), and How to Fix It

Four defects cause conflicts. Three can be fixed by transformations that keep the language; the fourth, ambiguity, needs a decision.

1. Left recursion

Technique: Removing immediate left recursion

$$A \to A\alpha_1 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \cdots \mid \beta_n$$
$$\Downarrow$$
$$A \to \beta_1 A' \mid \cdots \mid \beta_n A', \qquad A' \to \alpha_1 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon.$$

Both generate "one β, then any number of α's", the first growing the string to the left, the second to the right.

Why left recursion is fatal: $\mathrm{FIRST}(A\alpha) \supseteq \mathrm{FIRST}(\beta)$, so every cell of row A that selects β also selects $A\alpha$ (FIRST/FIRST conflict), and `A()` would call itself without consuming a token. **Associativity is not lost**: the tree of $E \to TE'$ leans right, but the semantic actions can still evaluate left to right; the grammar is a recogniser here.

When the exam forbids new nonterminals (mock 3.1(b)): a list rule $L \to L\,X \mid \varepsilon$ generates "zero or more X", which $L \to X\,L \mid \varepsilon$ generates too. Just flip the recursion.

Indirect left recursion ($S \to Aa \mid b$, $A \to Ac \mid Sd \mid \varepsilon$: $S \Rightarrow Aa \Rightarrow Sda$). Order the nonterminals (S, A); substitute earlier ones into later ones' leading positions, then remove the immediate recursion. Substituting S into $A \to Sd$: $A \to Ac \mid Aad \mid bd \mid \varepsilon$. Then $A \to bdA' \mid A'$, $A' \to cA' \mid adA' \mid \varepsilon$. The result has no left recursion *and is still not LL(1)* ($M[S, b]$ holds $S \to Aa$ and $S \to b$; $M[A', a]$ holds $A' \to adA'$ and $A' \to \varepsilon$): removing left recursion is necessary, not sufficient. Always rebuild the table.

2. Common prefixes: left factoring

Technique: Left factoring

$A \to \alpha\beta_1 \mid \alpha\beta_2 \quad \Longrightarrow \quad A \to \alpha A', \quad A' \to \beta_1 \mid \beta_2$. Postpone the choice until after the common prefix has been read.

The dangling else. $S \to \mathsf{if}\,C\,\mathsf{then}\,S \mid \mathsf{if}\,C\,\mathsf{then}\,S\,\mathsf{else}\,S \mid \mathsf{other}$ factors into $S \to \mathsf{if}\,C\,\mathsf{then}\,S\,S' \mid \mathsf{other}$, $S' \to \mathsf{else}\,S \mid \varepsilon$. Now $\mathrm{FOLLOW}(S') \supseteq \mathrm{FOLLOW}(S) \ni \mathsf{else}$, so $M[S', \mathsf{else}]$ holds both S' productions: the grammar is *ambiguous* (which if owns the else?), and factoring cannot cure ambiguity. Parsers resolve it by a decision: keep $S' \to \mathsf{else}\,S$ in that cell, so each else attaches to the nearest if.

3. ε-productions

Method: Eliminating ε-productions

(1) Find the nullable nonterminals. (2) For every production, add every version obtained by deleting any subset of its nullable occurrences (never produce a new $A \to \varepsilon$). (3) Delete all ε-productions. (4) If the start symbol was nullable, $\varepsilon \in L(G)$: add a new start $S' \to S \mid \varepsilon$.

This is not usually needed *for* LL(1) (LL(1) grammars are full of ε-productions, handled by FOLLOW), but it is a standard grammar transformation that exams ask for.

4. Ambiguity

No transformation removes ambiguity mechanically, and some languages have no unambiguous grammar at all. For expressions, use layering (§6); for the dangling else, a disambiguation decision.

Method: Is this grammar LL(1)? The checklist

(1) Ambiguous? Then no. (2) Left-recursive, directly or indirectly? Then no. (3) Two alternatives with a common prefix, or overlapping FIRST sets? Then no. (4) For each nullable alternative, does FIRST of the other alternatives meet FOLLOW(A)? Then no. Otherwise build the table: no double entry means yes. Justify with the specific cell and the two productions in it.

Mock drill

Mock 3.1. (a) **(C)**: *Stmts* $\to$ *Stmts Stmt* is left-recursive; on begin or id the parser cannot choose between it and *Stmts* $\to \varepsilon$ (both cells $M[\mathit{Stmts}, \mathsf{begin}]$ and $M[\mathit{Stmts}, \mathsf{id}]$ are doubly filled). Not (A): the grammar is unambiguous; not (B): *Block* starts with begin, *Simple* with id; not (D): it is given by a CFG. (b) Flip the recursion: *Stmts* $\to$ *Stmt Stmts* $\mid \varepsilon$. FIRST/FOLLOW (3.2): *Block* {begin} / {\$, begin, id, end}; *Stmts* {begin, id, ε} / {end}; *Stmt* and *Simple*: FIRST {begin, id} and {id}, FOLLOW {begin, id, end}.

Mock 1.6, $S \to (S)S \mid \varepsilon$. (a) Nullable: S. (b) Both occurrences of S in $(S)S$ may be deleted, giving four versions; $\varepsilon \in L$, so a new start:

$$S' \to S \mid \varepsilon, \qquad S \to (S)S \mid (S) \mid ()S \mid ().$$

Mock 1.4(d). $E \to E - T \mid T$ with $\alpha = -T$, $\beta = T$: $E \to TE'$, $E' \to -TE' \mid \varepsilon$.

The Night-Before Checklist

Summary: Everything in ten lines

1. Phases: lexer (regular, DFA) $\to$ parser (context-free, stack). CFG: one nonterminal on the left.
2. Regex precedence: star $>$ concatenation $>$ union. Test every example and say why each counterexample fails.
3. Regex $\to$ NFA (Thompson) $\to$ DFA (subsets, ε-closure of move) $\to$ minimal DFA (split groups). DFA can need 2^n states.
4. Not regular: infinitely many pairwise distinguishable strings ($a^n b^{2n}$, nested comments, balanced brackets).
5. Scanner: longest match, then first-listed rule; run until stuck, back up to the last accepting state; keywords before identifiers.
6. Leftmost derivation = top-down; one tree = one leftmost = one rightmost derivation; two trees = ambiguous.
7. Precedence by layers, associativity by the recursion side ($E \to E - T$ is left-associative).
8. FIRST: scan while nullable. FOLLOW: look at occurrences; add FIRST of what follows, and FOLLOW of the parent if that can vanish; \$ for the start; never ε.
9. Table: $A \to \alpha$ under FIRST(α), and under FOLLOW(A) if α is nullable. LL(1) $\iff$ no double cell. Stack: push the right side reversed; only matches advance.
10. Fixes: left recursion $\to$ $A \to \beta A'$, $A' \to \alpha A' \mid \varepsilon$ (or flip a list rule); left factor common prefixes; ambiguity needs a decision; remove ε by deleting nullable subsets and adding $S' \to S \mid \varepsilon$.